

PYTHON CONCEPTS & PROBLEM-SOLVING WORKBOOK

75 QUESTIONS ACROSS MULTIPLE PYTHON CONCEPT SOURCES

MCQs • 2-MARK QUESTIONS • PROBLEM SOLVING • REAL APPLICATIONS • PROGRAMMING THINKING

PURPOSE

Move from understanding to practice, problem solving, interview preparation and basic LeetCode readiness using only the concepts represented in the attached source files.

TABLE OF CONTENTS / INDEX

Use the links below to jump directly to a section. Every major question page also includes a Back to Index link.

SECTION A - 25 MCQs

SECTION B - 10 Two-Mark Questions

SECTION C - 20 Problem-Solving Questions

SECTION D - 10 Real-Application Questions

SECTION E - 10 Programming-Level Thinking Questions

SECTION F - Concept Notes

SECTION G - Answer Key & Explanations

APPENDIX - Question-to-Source Map

QUESTION DISTRIBUTION SUMMARY

SECTION	QUESTIONS
MCQs	25
2-Mark Questions	10
Problem Questions	20
Real Applications	10
Programming Thinking	10
TOTAL	75

SOURCE COVERAGE MATRIX

The attached sources were treated independently. The matrix below shows which concepts were used and how the 75 questions are distributed.

SOURCE FILE	CONCEPTS IDENTIFIED	MCQ	2-MARK	PROBLEM	REAL AP	THINK
s10.py	Modules, imports, aliases, import *; time module basics (localtime, sleep); regex password validation; user-defined m	3	1	2	1	1
s11.py	List comprehensions; ternary operator; nested ternary; lambda/anonymous functions; filter(); map(); chained map/filt	1	2	1	0	0
s12.py	requests GET requests; JSON response loading; API data extraction; file handling with open(); file modes; read/readlin	4	1	1	0	0
s13.py	json.loads/json.dumps; web scraping with requests + BeautifulSoup; selecting elements; text extraction; price parsing	3	1	2	0	0
s14(1).py	pathlib.Path; pandas read_csv; head/tail/columns; fillna/dropna; calculated columns; apply(lambda); duplicated/drop_	1	3	5	1	0
marks(1).csv	Sample student dataset with identity fields and subject columns; Total and Result fields; realistic values for data-anal	0	1	4	1	0

HOW TO USE THIS WORKBOOK

Attempt each question before reading the answer. Use the concept notes to review rules and common mistakes. For the later problems, focus on decomposition, edge cases and choosing the right Python tool.

MCQ

SECTION A

- 25 MCQs

Conceptual understanding, code tracing, debugging and source-grounded Python behavior.

[← BACK TO INDEX](#)

LEVEL ITEM 1 • Import Alias for a Function

DIFFICULTY: Easy	SOURCE: s10.py
CONCEPTS: imports, aliasing, user-defined modules	SKILL: MCQ

QUESTION

Suppose mymodule.py defines hi_msg(name). Which statement correctly imports it as hm?

- A. from mymodule import hi_msg as hm
- B. import hi_msg as hm from mymodule
- C. from hi_msg import mymodule as hm
- D. import mymodule.hi_msg as hm

[← BACK TO INDEX](#)

LEVEL ITEM 2 • `time.sleep()`

DIFFICULTY: Easy	SOURCE: s10.py
CONCEPTS: time module, sleep	SKILL: MCQ

QUESTION

What is the purpose of `sleep(5)` from the time module?

- A. Pause execution for about 5 seconds
- B. Restart the computer
- C. Close the Python file
- D. Convert seconds to hours

[← BACK TO INDEX](#)

LEVEL ITEM 3 • Regex Password Rule

DIFFICULTY: Medium	SOURCE: s10.py
CONCEPTS: re module, regular expressions, validation	SKILL: MCQ

QUESTION

In the password example, which regex check is used to require at least one digit?

- A. `re.search("[0-9]", pwd)`
- B. `re.search("[A-Z]", pwd)`
- C. `re.search("[a-z]", pwd)`
- D. `re.search("[@#$$%^&*]", pwd)`

[← BACK TO INDEX](#)

LEVEL ITEM 4 • List Comprehension Result

DIFFICULTY: Easy	SOURCE: s11.py
CONCEPTS: list comprehension	SKILL: MCQ

QUESTION

What does `[i if i % 2 == 0 else i ** 2 for i in [3,5,6]]` produce?

- A. [3, 5, 36]
- B. [9, 25, 6]
- C. [9, 5, 36]
- D. [3, 25, 6]

[← BACK TO INDEX](#)

LEVEL ITEM 5 • Ternary Operator

DIFFICULTY: Easy	SOURCE: s11.py
CONCEPTS: ternary operator	SKILL: MCQ

QUESTION

What is printed by `age=32; res = "You can vote" if age >= 18 else "You can't vote"; print(res)`?

- A. You can't vote
- B. You can vote
- C. 32
- D. True

[← BACK TO INDEX](#)

LEVEL ITEM 6 • Nested Ternary Logic

DIFFICULTY: Medium	SOURCE: s11.py
CONCEPTS: nested ternary, conditional expression	SKILL: MCQ

QUESTION

For num = -4, which label is produced by the nested ternary from the source?

- A. E & P
- B. E and N
- C. O and N
- D. E and Zero

[← BACK TO INDEX](#)

LEVEL ITEM 7 • lambda Function

DIFFICULTY: Easy	SOURCE: s11.py
CONCEPTS: lambda, anonymous function	SKILL: MCQ

QUESTION

What value does `(lambda name: f"Hello {name}")("Dev")` return?

- A. Dev
- B. Hello Dev
- C. Hello name
- D. None

[← BACK TO INDEX](#)

LEVEL ITEM 8 • filter() Semantics

DIFFICULTY: Medium	SOURCE: s11.py
CONCEPTS: filter, lambda	SKILL: MCQ

QUESTION

What remains after `list(filter(lambda n: n >= 20, [3, 35, 62, 2, 72]))`?

- A. [3, 35, 62, 2, 72]
- B. [35, 62, 72]
- C. [3, 2]
- D. [20, 35, 62, 72]

[← BACK TO INDEX](#)

LEVEL ITEM 9 • map() Semantics

DIFFICULTY: Medium	SOURCE: s11.py
CONCEPTS: map, lambda	SKILL: MCQ

QUESTION

What is the result of `list(map(lambda b: b**2, [2,3,4]))`?

- A. [2, 3, 4]
- B. [4, 9, 16]
- C. [2, 9, 8]
- D. [3, 6, 12]

[← BACK TO INDEX](#)

LEVEL ITEM 10 • Chained map/filter

DIFFICULTY: Hard	SOURCE: s11.py
CONCEPTS: map, filter, lambda	SKILL: Transformation pipeline

QUESTION

What does `list(map(lambda m:m**2, filter(lambda a:a>=20, [3,35,62,2,72,18,9])))` return?

- A. [1225, 3844, 5184]
- B. [9, 1225, 3844, 5184]
- C. [35, 62, 72]
- D. [3,35,62,72]

[← BACK TO INDEX](#)

LEVEL ITEM 11 • requests.get()

DIFFICULTY: Easy	SOURCE: s12.py
CONCEPTS: requests, HTTP GET	SKILL: MCQ

QUESTION

What is the purpose of `requests.get(url)` in the API example?

- A. Returns nothing
- B. Sends a GET request and returns a Response object
- C. Parses JSON
- D. Writes a file

[← BACK TO INDEX](#)

LEVEL ITEM 12 • JSON loading

DIFFICULTY: Medium	SOURCE: s12.py
CONCEPTS: json.loads, JSON parsing	SKILL: MCQ

QUESTION

If response.text contains JSON text, which function converts it into Python objects?

- A. json.dumps(response.text)
- B. json.loads(response.text)
- C. requests.load(response.text)
- D. json.parse(response.text)

[← BACK TO INDEX](#)

LEVEL ITEM 13 • File mode w

DIFFICULTY: Easy	SOURCE: s12.py
CONCEPTS: open, file modes	SKILL: MCQ

QUESTION

What does `open("demo.txt", "w")` do?

- A. Read only
- B. Write
- C. Append only
- D. Binary read only

[← BACK TO INDEX](#)

LEVEL ITEM 14 • readline()

DIFFICULTY: Easy	SOURCE: s12.py
CONCEPTS: file handling, readline	SKILL: MCQ

QUESTION

What does `f.readline()` return?

- A. One line from the file
- B. The entire file as a string
- C. A list of all lines
- D. A single character

[← BACK TO INDEX](#)

LEVEL ITEM 15 • json.dumps()

DIFFICULTY: Medium	SOURCE: s13.py
CONCEPTS: json.dumps, serialization	SKILL: MCQ

QUESTION

What is json.dumps() used for?

- A. Python object -> JSON text
- B. JSON text -> Python object
- C. Python list -> CSV
- D. HTML -> DOM

[← BACK TO INDEX](#)

LEVEL ITEM 16 • BeautifulSoup parser

DIFFICULTY: Medium	SOURCE: s13.py
CONCEPTS: BeautifulSoup, web scraping	SKILL: MCQ

QUESTION

Which parser is used in the source when constructing BeautifulSoup?

- A. lxml
- B. html.parser
- C. xml
- D. json.parser

[← BACK TO INDEX](#)

LEVEL ITEM 17 • Price parsing

DIFFICULTY: Medium	SOURCE: s13.py
CONCEPTS: web scraping, string replace, float	SKILL: MCQ

QUESTION

Why does the price extraction code call `replace("₹", "").replace(", ", "")` before `float(k)`?

- A. To make the text shorter
- B. To remove formatting that prevents numeric conversion
- C. To convert float to string
- D. To find HTML tags

[← BACK TO INDEX](#)

LEVEL ITEM 18 • Path(__file__).parent

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pathlib, Path	SKILL: MCQ

QUESTION

What does Path(__file__).parent represent in the CSV example?

- A. The current directory of the shell
- B. The directory containing the script
- C. The root of the filesystem
- D. The marks column

[← BACK TO INDEX](#)

LEVEL ITEM 19 • pandas read_csv

DIFFICULTY: Easy	SOURCE: s14(1).py
CONCEPTS: pandas, read_csv	SKILL: MCQ

QUESTION

Which pandas call loads the marks CSV into a DataFrame?

- A. `pd.read_csv(csv_file)`
- B. `pd.load_csv(csv_file)`
- C. `pd.open_csv(csv_file)`
- D. `pd.import_csv(csv_file)`

[← BACK TO INDEX](#)

LEVEL ITEM 20 • head() vs tail()

DIFFICULTY: Easy	SOURCE: s14(1).py
CONCEPTS: pandas, head, tail	SKILL: MCQ

QUESTION

What is the difference between head(2) and tail(8)?

- A. head(2)=last 2; tail(8)=first 8
- B. head(2)=first 2; tail(8)=last 8
- C. Both show first rows
- D. Both show last rows

[← BACK TO INDEX](#)

LEVEL ITEM 21 • fillna vs dropna

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pandas, missing data	SKILL: MCQ

QUESTION

Which statement is correct?

- A. fillna removes rows; dropna fills values
- B. fillna fills missing values; dropna removes rows
- C. Both do the same thing
- D. fillna sorts rows; dropna renames columns

[← BACK TO INDEX](#)

LEVEL ITEM 22 • apply lambda

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pandas, apply, lambda	SKILL: MCQ

QUESTION

What does `data["Total"].apply(lambda n: "pass" if n > 200 else "fail")` do?

- A. Creates a pass/fail label per Total value
- B. Sorts Total
- C. Reads the CSV
- D. Drops nulls

[← BACK TO INDEX](#)

LEVEL ITEM 23 • loc condition

DIFFICULTY: Hard	SOURCE: s14(1).py
CONCEPTS: pandas, loc, boolean filtering	SKILL: MCQ

QUESTION

Which expression filters rows where Maths > 30 and Result == "fail"?

- A. `data.loc[(data["Maths"] > 30) & (data["Result"] == "fail")]`
- B. `data.loc[data["Maths"] > 30 and data["Result"] == "fail"]`
- C. `data.iloc[(data["Maths"] > 30) & (data["Result"] == "fail")]`
- D. `data.loc[data["Maths"] > 30 | data["Result"] == "fail"]`

[← BACK TO INDEX](#)

LEVEL ITEM 24 • `iloc` slicing

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pandas, <code>iloc</code>	SKILL: MCQ

QUESTION

For a DataFrame, what does `data.iloc[2:6]` include?

- A. Rows 2,3,4,5
- B. Rows 2,3,4,5,6
- C. Rows 0,1,2,3,4,5
- D. Only row 6

[← BACK TO INDEX](#)

LEVEL ITEM 25 • drop_duplicates()

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pandas, duplicates	SKILL: MCQ

QUESTION

What is the purpose of drop_duplicates()?

- A. Adds duplicate rows
- B. Removes duplicate rows
- C. Finds nulls
- D. Sorts rows

[← BACK TO INDEX](#)

2-MARK

SECTION B

- 10 TWO-MARK QUESTIONS

Short class-test style questions focused on direct concept application.

[← BACK TO INDEX](#)

LEVEL ITEM 26 • Explain a Module Alias

DIFFICULTY: Easy	SOURCE: s10.py
CONCEPTS: modules, aliases	SKILL: 2-MARK

PROBLEM STATEMENT

Write one line that imports `hello_msg` from `mymodule` and gives it the alias `hm`.

REQUIREMENTS

- Use `from ... import ... as ...` syntax.

EXAMPLE INPUT

EXPECTED OUTPUT

```
from mymodule import hello_msg as hm
```

THINKING HINT

Recall the alias pattern.

[← BACK TO INDEX](#)

LEVEL ITEM 27 • List Comprehension Rewrite

DIFFICULTY: Easy	SOURCE: s11.py
CONCEPTS: list comprehension	SKILL: 2-MARK

PROBLEM STATEMENT

Rewrite a loop that appends the numbers 1 to 5 into a single list-comprehension expression.

REQUIREMENTS

EXAMPLE INPUT

```
for i in range(1,6): append i
```

EXPECTED OUTPUT

```
li = [i for i in range(1, 6)]
```

THINKING HINT

Move the expression before the for clause.

[← BACK TO INDEX](#)

LEVEL ITEM 28 • Ternary Conversion

DIFFICULTY: Easy	SOURCE: s11.py
CONCEPTS: ternary operator	SKILL: 2-MARK

PROBLEM STATEMENT

Convert: if age >= 18: result="adult" else: result="minor" into a one-line conditional expression.

REQUIREMENTS

EXAMPLE INPUT

EXPECTED OUTPUT

```
result = "adult" if age >= 18 else "minor"
```

THINKING HINT

Use value_if_true if condition else value_if_false.

[← BACK TO INDEX](#)

LEVEL ITEM 29 • filter() Task

DIFFICULTY: Easy	SOURCE: s11.py
CONCEPTS: filter, lambda	SKILL: 2-MARK

PROBLEM STATEMENT

Write one line to keep only values ≥ 10 from [4,10,15,2].

REQUIREMENTS

EXAMPLE INPUT

EXPECTED OUTPUT

```
list(filter(lambda x: x >= 10, [4,10,15,2]))
```

THINKING HINT

filter keeps items whose predicate is true.

[← BACK TO INDEX](#)

LEVEL ITEM 30 • Write a File

DIFFICULTY: Easy	SOURCE: s12.py
CONCEPTS: file handling, write	SKILL: 2-MARK

PROBLEM STATEMENT

Write two lines, "A" and "B", to demo.txt using one open call in write mode.

REQUIREMENTS

EXAMPLE INPUT

EXPECTED OUTPUT

```
f = open("demo.txt","w"); f.write("A\nB\n"); f.close()
```

THINKING HINT

Use write mode and newline characters.

[← BACK TO INDEX](#)

LEVEL ITEM 31 • JSON Conversion

DIFFICULTY: Medium	SOURCE: s13.py
CONCEPTS: json.loads, json.dumps	SKILL: 2-MARK

PROBLEM STATEMENT

State the two Python/json conversion operations represented by loads and dumps.

REQUIREMENTS

EXAMPLE INPUT

EXPECTED OUTPUT

loads: JSON text -> Python object; dumps: Python object -> JSON text

THINKING HINT

Think parsing vs serialization.

[← BACK TO INDEX](#)

LEVEL ITEM 32 • Scrape One Field

DIFFICULTY: Medium	SOURCE: s13.py
CONCEPTS: requests, BeautifulSoup	SKILL: 2-MARK

PROBLEM STATEMENT

Write the two main steps needed to turn an HTTP response into a BeautifulSoup object.

REQUIREMENTS

EXAMPLE INPUT

EXPECTED OUTPUT

```
response = requests.get(url); soup = BeautifulSoup(response.text, "html.parser")
```

THINKING HINT

Fetch first, parse second.

[← BACK TO INDEX](#)

LEVEL ITEM 33 • Read First Two Rows

DIFFICULTY: Easy	SOURCE: s14(1).py
CONCEPTS: pandas, head	SKILL: 2-MARK

PROBLEM STATEMENT

Write the pandas statement that prints the first two rows of data.

REQUIREMENTS

EXAMPLE INPUT

EXPECTED OUTPUT

```
print(data.head(2))
```

THINKING HINT

Use head with a row count.

[← BACK TO INDEX](#)

LEVEL ITEM 34 • Create Total Column

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pandas, calculated column	SKILL: 2-MARK

PROBLEM STATEMENT

Write a statement that creates Total from Maths + Physics + Chemistry.

REQUIREMENTS

EXAMPLE INPUT

EXPECTED OUTPUT

```
data["Total"] = data["Maths"] + data["Physics"] + data["Chemistry"]
```

THINKING HINT

Use vectorized Series addition.

[← BACK TO INDEX](#)

LEVEL ITEM 35 • Filter Pass Rows

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pandas, loc, boolean filtering	SKILL: 2-MARK

PROBLEM STATEMENT

Write a `.loc` expression to select rows where Result equals "pass".

REQUIREMENTS

EXAMPLE INPUT

--

EXPECTED OUTPUT

```
data.loc[data["Result"] == "pass"]
```

THINKING HINT

Use `.loc` with a boolean Series.

[← BACK TO INDEX](#)

SECTION C

- 20 PROBLEM-SOLVING QUESTIONS

Easy to hard practical coding tasks built from the attached concepts.

← BACK TO INDEX

LEVEL ITEM 36 • Create a reusable password validator

DIFFICULTY: Easy	SOURCE: s10.py
CONCEPTS: re, regex, loop, validation	SKILL: PROBLEM

PROBLEM STATEMENT

Write a validator that accepts a password and checks the same categories shown in the source: length bounds plus uppercase, lowercase, digit and special character.

REQUIREMENTS

- Treat a password as valid only when all required categories are present.
- Return/print a clear Valid or Invalid result.

EXAMPLE INPUT

Input: "Dev@1234x"

EXPECTED OUTPUT

Output: Valid

THINKING HINT

Check one requirement at a time; a boolean flag or early-failure approach is enough.

← BACK TO INDEX

LEVEL ITEM 37 • Module menu wrapper

DIFFICULTY: Medium	SOURCE: s10.py
CONCEPTS: user-defined modules, imports, functions	SKILL: PROBLEM

PROBLEM STATEMENT

Design a small program where `mymodule.py` contains `hello_msg()` and `hi_msg()`, and a main script imports both functions and calls them based on a menu choice.

REQUIREMENTS

- Use `from mymodule import ...`
- Do not duplicate the function definitions in the main script.

EXAMPLE INPUT

Choice: 1; Name: Dev

EXPECTED OUTPUT

Output: Hello Dev

THINKING HINT

Think of the module as the library and the main file as the caller.

[← BACK TO INDEX](#)

LEVEL ITEM 38 • Transform mixed values with a lambda

DIFFICULTY: Easy	SOURCE: s11.py
CONCEPTS: lambda, conditional expression	SKILL: PROBLEM

PROBLEM STATEMENT

Given [3, 4, 7, 8], create a new list where even numbers stay unchanged and odd numbers are squared.

REQUIREMENTS

- Use a lambda inside map().

EXAMPLE INPUT

Input: [3,4,7,8]

EXPECTED OUTPUT

Output: [9,4,49,8]

THINKING HINT

The source already shows lambda + conditional transformation.

← BACK TO INDEX

LEVEL ITEM 39 • Filter then transform

DIFFICULTY: Easy	SOURCE: s11.py
CONCEPTS: filter, map, lambda	SKILL: PROBLEM

PROBLEM STATEMENT

Keep values ≥ 20 from [3,35,62,2,72,18,9], then square them.

REQUIREMENTS

- Use filter followed by map.

EXAMPLE INPUT

Input: [3,35,62,2,72,18,9]

EXPECTED OUTPUT

Output: [1225,3844,5184]

THINKING HINT

Build the pipeline in two steps before nesting it.

[← BACK TO INDEX](#)

LEVEL ITEM 40 • Build numbers 1-100 with comprehension

DIFFICULTY: Easy	SOURCE: s11.py
CONCEPTS: list comprehension, range	SKILL: PROBLEM

PROBLEM STATEMENT

Create a list from 1 to 100 using a single list comprehension.

REQUIREMENTS

- Do not use `append()`.

EXAMPLE INPUT

Input: none

EXPECTED OUTPUT

Output begins [1,2,3,...,100]

THINKING HINT

Translate “for each `i` in range” directly into comprehension syntax.

[← BACK TO INDEX](#)

LEVEL ITEM 41 • Nested ternary classifier

DIFFICULTY: Medium	SOURCE: s11.py
CONCEPTS: nested ternary, modulus, conditions	SKILL: PROBLEM

PROBLEM STATEMENT

Classify an integer as Even/Odd and Positive/Negative/Zero. Use a nested conditional expression.

REQUIREMENTS

- Zero should be classified separately.

EXAMPLE INPUT

Input: -8

EXPECTED OUTPUT

Output: Even and Negative

THINKING HINT

Design the categories first, then encode them compactly.

← BACK TO INDEX

LEVEL ITEM 42 • Extract API titles

DIFFICULTY: Medium	SOURCE: s12.py
CONCEPTS: requests, json.loads, map	SKILL: PROBLEM

PROBLEM STATEMENT

Given a JSON response string containing product dictionaries, parse it and print a list of product titles.

REQUIREMENTS

- Use json.loads and map.

EXAMPLE INPUT

```
Input JSON: [{"title":"A"}, {"title":"B"}]
```

EXPECTED OUTPUT

```
Output: ["A", "B"]
```

THINKING HINT

Parse first, then map over dictionaries to select title.

[← BACK TO INDEX](#)

LEVEL ITEM 43 • Read a text file safely

DIFFICULTY: Easy	SOURCE: s12.py
CONCEPTS: open, read, close	SKILL: PROBLEM

PROBLEM STATEMENT

Read the full contents of a text file and print them.

REQUIREMENTS

- Open in read mode and close the file.

EXAMPLE INPUT

```
demo.txt contains: Hello  
Python
```

EXPECTED OUTPUT

```
Output: Hello then Python
```

THINKING HINT

Use `read()` for the entire remaining file content.

[← BACK TO INDEX](#)

LEVEL ITEM 44 • Append a log line

DIFFICULTY: Easy	SOURCE: s12.py
CONCEPTS: open, append mode	SKILL: PROBLEM

PROBLEM STATEMENT

Add a new line to an existing log file without deleting earlier content.

REQUIREMENTS

- Use append mode.
- Write a trailing newline.

EXAMPLE INPUT

```
Existing: Start  
Append: Success
```

EXPECTED OUTPUT

```
Output file lines: Start / Success
```

THINKING HINT

Append mode is “add to the end”.

[← BACK TO INDEX](#)

LEVEL ITEM 45 • Write rows from a list

DIFFICULTY: Medium	SOURCE: s12.py
CONCEPTS: open, write, loop	SKILL: PROBLEM

PROBLEM STATEMENT

Write every item of ["a","b","c","d"] on its own line in demo.txt.

REQUIREMENTS

- Use a loop and f.write().

EXAMPLE INPUT

```
Input list: a,b,c,d
```

EXPECTED OUTPUT

```
demo.txt contains four lines.
```

THINKING HINT

The source performs this exact pattern in a loop.

[← BACK TO INDEX](#)

LEVEL ITEM 46 • Choose read method

DIFFICULTY: Medium	SOURCE: s12.py
CONCEPTS: read, readline, readlines	SKILL: PROBLEM

PROBLEM STATEMENT

Write a small program that prints the first line, then prints the remaining lines as a list.

REQUIREMENTS

- Use `readline()` once, then `readlines()`.

EXAMPLE INPUT

```
File: A
B
C
```

EXPECTED OUTPUT

```
Output: first line A, remaining [B,C]
```

THINKING HINT

Remember that the file pointer advances after `readline()`.

[← BACK TO INDEX](#)

LEVEL ITEM 47 • Convert Python data to JSON

DIFFICULTY: Easy	SOURCE: s13.py
CONCEPTS: json.dumps, serialization	SKILL: PROBLEM

PROBLEM STATEMENT

Serialize [{"name": "Dev"}] into a JSON string and print it.

REQUIREMENTS

- Use json.dumps.

EXAMPLE INPUT

Input object: [{"name": "Dev"}]

EXPECTED OUTPUT

Output is a JSON string representation.

THINKING HINT

The source contrasts a Python list of dicts with JSON text.

[← BACK TO INDEX](#)

LEVEL ITEM 48 • Scrape titles from HTML

DIFFICULTY: Medium	SOURCE: s13.py
CONCEPTS: requests, BeautifulSoup, find_all	SKILL: PROBLEM

PROBLEM STATEMENT

Given an HTML string containing elements with class product-title, extract their text values.

REQUIREMENTS

- Use BeautifulSoup and find_all.
- Return a list of titles.

EXAMPLE INPUT

HTML with two product-title divs

EXPECTED OUTPUT

```
["Shoe A", "Shoe B"]
```

THINKING HINT

Find all matching elements, then access .text.

[← BACK TO INDEX](#)

LEVEL ITEM 49 • Filter scraped prices

DIFFICULTY: Medium	SOURCE: s13.py
CONCEPTS: BeautifulSoup, replace, float, conditions	SKILL: PROBLEM

PROBLEM STATEMENT

From price strings like ₹17,500, print only prices between 17000 and 20000 inclusive.

REQUIREMENTS

- Strip ₹ and commas before float conversion.

EXAMPLE INPUT

```
["₹16,500", "₹17,500", "₹20,000", "₹21,000"]
```

EXPECTED OUTPUT

```
₹17,500 and ₹20,000
```

THINKING HINT

Treat the formatted currency as text first, numeric comparison second.

[← BACK TO INDEX](#)

LEVEL ITEM 50 • Save scraped titles to a file

DIFFICULTY: Hard	SOURCE: s13.py
CONCEPTS: web scraping, file handling	SKILL: PROBLEM

PROBLEM STATEMENT

Write a list of scraped titles to demo.txt with numbered lines 1., 2., 3. ...

REQUIREMENTS

- Use a counter or enumerate.
- One title per line.

EXAMPLE INPUT

```
Titles: ["A","B","C"]
```

EXPECTED OUTPUT

```
File lines: 1.A / 2.B / 3.C
```

THINKING HINT

Combine scraping-style text extraction with the file-writing pattern.

[← BACK TO INDEX](#)

LEVEL ITEM 51 • Build Total and Result columns

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pandas, calculated columns, apply	SKILL: PROBLEM

PROBLEM STATEMENT

Given a DataFrame with Maths, Physics and Chemistry, create Total and Result where Result is pass if Total > 200 else fail.

REQUIREMENTS

- Use vectorized addition and Series.apply with a lambda.

EXAMPLE INPUT

Rows with subject marks

EXPECTED OUTPUT

Output includes Total and Result columns

THINKING HINT

Build Total first, then classify it.

[← BACK TO INDEX](#)

LEVEL ITEM 52 • Handle missing values

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pandas, fillna, dropna	SKILL: PROBLEM

PROBLEM STATEMENT

Given a DataFrame with missing values, demonstrate both `fillna(0)` and `dropna()`, then explain the difference by printing the resulting row counts.

REQUIREMENTS

- Do not mix up replacement and removal.

EXAMPLE INPUT

DataFrame with some NaN values

EXPECTED OUTPUT

Two outputs with different row counts

THINKING HINT

Count rows before and after each strategy.

[← BACK TO INDEX](#)

LEVEL ITEM 53 • Inspect a DataFrame

DIFFICULTY: Easy	SOURCE: s14(1).py
CONCEPTS: pandas, head, tail, columns	SKILL: PROBLEM

PROBLEM STATEMENT

Write a report that prints the first 2 rows, last 2 rows, and column names of the marks DataFrame.

REQUIREMENTS

- Use `head(2)`, `tail(2)`, and `columns.values`.

EXAMPLE INPUT

```
marks.csv
```

EXPECTED OUTPUT

```
The report contains all three pieces.
```

THINKING HINT

Break the task into three independent inspection operations.

[← BACK TO INDEX](#)

LEVEL ITEM 54 • Filter with loc

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pandas, loc, boolean conditions	SKILL: PROBLEM

PROBLEM STATEMENT

From marks.csv, select rows where Maths > 30 and Result == "fail".

REQUIREMENTS

- Use .loc and & for the combined condition.

EXAMPLE INPUT

```
marks.csv
```

EXPECTED OUTPUT

```
Only rows meeting both conditions remain.
```

THINKING HINT

Build each boolean condition separately if needed.

[← BACK TO INDEX](#)

LEVEL ITEM 55 • Combine map/filter with student totals

DIFFICULTY: Hard	SOURCE: s11.py + s14(1).py + marks(1).csv
CONCEPTS: map, filter, pandas, lambda	SKILL: PROBLEM

PROBLEM STATEMENT

Create a list of students whose Total is above 210, then map those rows to strings formatted as "Name - Total".

REQUIREMENTS

- Use a Python-level filter/map pipeline after extracting the relevant records.

EXAMPLE INPUT

```
marks.csv
```

EXPECTED OUTPUT

Example output contains Suresh - 249, Jessica - 260, Jennifer - 232, Ritesh - 217, Farha - 207? (exclud

THINKING HINT

First decide which rows survive, then transform each survivor.

← BACK TO INDEX

SECTION D

- 10 REAL-APPLICATION QUESTIONS

Real-world scenarios showing where the source concepts fit in applications.

← BACK TO INDEX

LEVEL ITEM 56 • Password Gate for an Admin Portal

DIFFICULTY: Advanced	SOURCE: s10.py
CONCEPTS: re, validation, loop, sleep	SKILL: REAL

REAL-WORLD SCENARIO

You are building a login guard that temporarily pauses after three consecutive wrong password attempts.

REQUIREMENTS

- Validate passwords using the same character requirements as the source.
- Allow limited retries.
- Use `sleep()` to model the pause.
- Print a clear success or lock message.

WHY THIS CONCEPT IS USED

The source combines regex validation, loop control, and sleep; together these resemble an account-protection flow.

EXAMPLE INPUT

Input: password attempts; policy from source

EXPECTED OUTPUT

Output: success, retry, or temporary pause

THINKING HINT

Design separate validation and retry responsibilities.

[← BACK TO INDEX](#)

LEVEL ITEM 57 • API Product Title Extractor

DIFFICULTY: Advanced	SOURCE: s12.py
CONCEPTS: requests, json, map	SKILL: REAL

REAL-WORLD SCENARIO

A retail dashboard receives product JSON from an API and needs a list of product titles for a UI.

REQUIREMENTS

- Fetch/parse product JSON.
- Extract title from every product.
- Return a clean list.
- Handle an empty product list.

WHY THIS CONCEPT IS USED

This application maps directly to the requests/API and JSON extraction pattern demonstrated in the source. Network availability would be required in a live app.

EXAMPLE INPUT

API response shaped like list of product dictionaries

EXPECTED OUTPUT

A Python list of titles

THINKING HINT

Treat network parsing and transformation as separate steps.

← BACK TO INDEX

LEVEL ITEM 58 • Audit trail writer

DIFFICULTY: Medium	SOURCE: s12.py
CONCEPTS: open, append, file handling	SKILL: REAL

REAL-WORLD SCENARIO

A service needs to append one audit event per line without erasing previous events.

REQUIREMENTS

- Open the file in append mode.
- Write timestamp/message text.
- Close the file.

WHY THIS CONCEPT IS USED

The source explicitly distinguishes write and append modes. This is a direct real-world use of append logging.

EXAMPLE INPUT

```
Existing log plus new event
```

EXPECTED OUTPUT

```
Old entries remain; new entry appears at the end
```

THINKING HINT

Use append rather than write mode.

[← BACK TO INDEX](#)

LEVEL ITEM 59 • Product Catalog Scraper

DIFFICULTY: Advanced	SOURCE: s13.py
CONCEPTS: requests, BeautifulSoup, file writing	SKILL: REAL

REAL-WORLD SCENARIO

A procurement team wants to collect product names from a webpage and save them to a text file.

REQUIREMENTS

- Request the webpage.
- Parse with BeautifulSoup.
- Find matching product nodes.
- Write a numbered list to a file.

WHY THIS CONCEPT IS USED

The source shows the same pipeline: requests/HTML -> BeautifulSoup -> text extraction -> file output.

EXAMPLE INPUT

Web page HTML

EXPECTED OUTPUT

product_catalog.txt contains numbered titles

THINKING HINT

Separate fetch, parse, extract, persist.

[← BACK TO INDEX](#)

LEVEL ITEM 60 • Price-band monitoring

DIFFICULTY: Advanced	SOURCE: s13.py
CONCEPTS: BeautifulSoup, replace, float, conditions	SKILL: REAL

REAL-WORLD SCENARIO

A buyer wants only products priced between 17,000 and 20,000. Parse formatted currency strings and report matches.

REQUIREMENTS

- Strip currency symbol and commas.
- Convert to float.
- Keep values in the inclusive range.

WHY THIS CONCEPT IS USED

This is a direct operational use of the price-cleaning and filtering logic in the source.

EXAMPLE INPUT

Price strings such as ₹17,500

EXPECTED OUTPUT

Matching prices only

THINKING HINT

Normalize text before numeric comparison.

← BACK TO INDEX

LEVEL ITEM 61 • Student result pipeline

DIFFICULTY: Advanced	SOURCE: s14(1).py + marks(1).csv
CONCEPTS: pandas, fillna, dropna, apply	SKILL: REAL

REAL-WORLD SCENARIO

Build a mini result pipeline that loads the supplied marks dataset, cleans missing values, computes Total, assigns Result, and prints a compact report.

REQUIREMENTS

- Read the CSV.
- Handle missing values.
- Compute Total from Maths/Physics/Chemistry.
- Classify pass/fail using Total > 200.
- Print Name, Total and Result.

WHY THIS CONCEPT IS USED

The complete workflow is directly supported by s14.py and the supplied CSV.

EXAMPLE INPUT

```
marks.csv
```

EXPECTED OUTPUT

```
A compact result report for all students
```

THINKING HINT

Follow a deterministic pipeline: load -> clean -> derive -> classify -> report.

[← BACK TO INDEX](#)

LEVEL ITEM 62 • Data-quality cleanup job

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pandas, duplicated, drop_duplicates, dropna	SKILL: REAL

REAL-WORLD SCENARIO

Create a cleanup routine for an imported DataFrame that reports duplicate rows, removes them, drops incomplete rows, and prints the final row count.

REQUIREMENTS

- Show duplicated() results.
- Apply drop_duplicates().
- Apply dropna().
- Print final number of rows.

WHY THIS CONCEPT IS USED

This uses only the data-cleaning operations explicitly shown in the source.

EXAMPLE INPUT

Any DataFrame with duplicate/missing records

EXPECTED OUTPUT

Final cleaned row count

THINKING HINT

Keep detection and transformation steps explicit.

← BACK TO INDEX

LEVEL ITEM 63 • CSV segment viewer

DIFFICULTY: Medium	SOURCE: s14(1).py + marks(1).csv
CONCEPTS: pandas, iloc, loc	SKILL: REAL

REAL-WORLD SCENARIO

Create a viewer that can show a positional slice of rows or a condition-based subset from the supplied dataset.

REQUIREMENTS

- Support one mode for iloc slicing.
- Support one mode for loc filtering.
- Do not change the original DataFrame.

WHY THIS CONCEPT IS USED

The source explicitly contrasts iloc slicing with loc conditional filtering.

EXAMPLE INPUT

Mode + parameters

EXPECTED OUTPUT

A displayed DataFrame subset

THINKING HINT

Choose the correct selector based on the request: positional vs conditional.

[← BACK TO INDEX](#)

LEVEL ITEM 64 • Exportable performance report

DIFFICULTY: Hard	SOURCE: s14(1).py + marks(1).csv
CONCEPTS: pandas, to_csv, sort, CSV	SKILL: REAL

REAL-WORLD SCENARIO

Create a report sorted by Total descending and save it to a new CSV file.

REQUIREMENTS

- Load the supplied dataset.
- Sort by Total descending.
- Save with to_csv.
- Keep the original CSV untouched.

WHY THIS CONCEPT IS USED

Sorting is a reasonable reporting extension using pandas plus the supplied data; persistence uses the source to_csv concept.

EXAMPLE INPUT

```
marks.csv
```

EXPECTED OUTPUT

```
A new CSV file containing sorted records
```

THINKING HINT

Sort first, then persist the transformed DataFrame.

[← BACK TO INDEX](#)

LEVEL ITEM 65 • Functional report generator

DIFFICULTY: Hard	SOURCE: s11.py + s14(1).py + marks(1).csv
CONCEPTS: map, filter, pandas, CSV	SKILL: REAL

REAL-WORLD SCENARIO

Create a report of students with Total ≥ 200 and format each row as "Name: pass/fail" using a filter/map pipeline.

REQUIREMENTS

- Load the CSV with pandas.
- Convert rows to records.
- Filter qualifying totals.
- Map records to formatted strings.

WHY THIS CONCEPT IS USED

This intentionally combines the functional programming concepts in s11.py with the tabular processing shown in s14.py.

EXAMPLE INPUT

```
marks.csv
```

EXPECTED OUTPUT

A list of formatted report strings

THINKING HINT

Convert tabular records to Python dictionaries, then use the functional pipeline.

← BACK TO INDEX

THIN10 PROG RAMMING- LEVEL THINKING

Higher-order reasoning tasks emphasizing decomposition, data choice and efficient thinking.

← BACK TO INDEX

LEVEL ITEM 66 • Choose map, filter or both

DIFFICULTY: Hard	SOURCE: s11.py
CONCEPTS: map, filter, lambda	SKILL: Filtering + transformation

SCENARIO

You receive [12, 7, 25, 4, 30]. The business rule says: keep values ≥ 10 , then square the survivors. Decide which functional operations are necessary and justify the order.

CHALLENGE

You receive [12, 7, 25, 4, 30]. The business rule says: keep values ≥ 10 , then square the survivors. Decide which functional operations are necessary and justify the order.

THINKING REQUIREMENTS

- Identify what data must be stored.
- Choose the most appropriate Python concepts from the attached sources.
- State at least one edge case to handle.

REQUIREMENTS

EXAMPLE INPUT

Input list

EXPECTED OUTPUT

Output [144,625,900]

THINKING HINT

Ask: am I selecting items, transforming items, or both?

INTERVIEW / LEETCODE SKILL

Filtering + transformation

[← BACK TO INDEX](#)

LEVEL ITEM 67 • Design a module boundary

DIFFICULTY: Hard	SOURCE: s10.py
CONCEPTS: user-defined modules, imports, reusability	SKILL: THINK

SCENARIO

You have three related helper functions: greeting, password validation, and formatting. Decide which should live in mymodule.py and how the main file should import them.

CHALLENGE

You have three related helper functions: greeting, password validation, and formatting. Decide which should live in mymodule.py and how the main file should import them.

THINKING REQUIREMENTS

- Identify what data must be stored.
- Choose the most appropriate Python concepts from the attached sources.
- State at least one edge case to handle.

REQUIREMENTS

EXAMPLE INPUT

Three functions

EXPECTED OUTPUT

A proposed module/main structure

THINKING HINT

Separate reusable behavior from user interaction.

← BACK TO INDEX

LEVEL ITEM 68 • Make a file workflow robust

DIFFICULTY: Hard	SOURCE: s12.py
CONCEPTS: file handling, read, append, close	SKILL: THINK

SCENARIO

A program reads a config file, appends one new line, then reads the file again. Explain how the file pointer and open modes should be handled.

CHALLENGE

A program reads a config file, appends one new line, then reads the file again. Explain how the file pointer and open modes should be handled.

THINKING REQUIREMENTS

- Identify what data must be stored.
- Choose the most appropriate Python concepts from the attached sources.
- State at least one edge case to handle.

REQUIREMENTS

EXAMPLE INPUT

A text file

EXPECTED OUTPUT

Final content includes original plus appended line

THINKING HINT

Think in separate open/read and open/append phases.

← BACK TO INDEX

LEVEL ITEM 69 • Parse once, use many times

DIFFICULTY: Medium	SOURCE: s12.py + s13.py
CONCEPTS: json.loads, JSON, data access	SKILL: THINK

SCENARIO

A service returns JSON text containing many product dictionaries. Design a flow that parses the text once and then extracts titles and prices without reparsing.

CHALLENGE

A service returns JSON text containing many product dictionaries. Design a flow that parses the text once and then extracts titles and prices without reparsing.

THINKING REQUIREMENTS

- Identify what data must be stored.
- Choose the most appropriate Python concepts from the attached sources.
- State at least one edge case to handle.

REQUIREMENTS

EXAMPLE INPUT

JSON string

EXPECTED OUTPUT

Two Python lists: titles and prices

THINKING HINT

Separate parsing from transformation.

← BACK TO INDEX

LEVEL ITEM 70 • Scraping pipeline failure points

DIFFICULTY: Hard	SOURCE: s13.py
CONCEPTS: requests, BeautifulSoup, file handling	SKILL: THINK

SCENARIO

A scraper returns an empty title list. Identify three likely checkpoints to inspect before changing the rest of the program.

CHALLENGE

A scraper returns an empty title list. Identify three likely checkpoints to inspect before changing the rest of the program.

THINKING REQUIREMENTS

- Identify what data must be stored.
- Choose the most appropriate Python concepts from the attached sources.
- State at least one edge case to handle.

REQUIREMENTS

EXAMPLE INPUT

HTML page + scraper code

EXPECTED OUTPUT

Three diagnostic checks

THINKING HINT

Think request, parser, selector.

← BACK TO INDEX

LEVEL ITEM 71 • Pick loc vs iloc

DIFFICULTY: Medium	SOURCE: s14(1).py
CONCEPTS: pandas, loc, iloc	SKILL: THINK

SCENARIO

You need rows 2 through 5 by position, and separately all rows where Maths > 70. Decide whether each task should use iloc or loc and explain why.

CHALLENGE

You need rows 2 through 5 by position, and separately all rows where Maths > 70. Decide whether each task should use iloc or loc and explain why.

THINKING REQUIREMENTS

- Identify what data must be stored.
- Choose the most appropriate Python concepts from the attached sources.
- State at least one edge case to handle.

REQUIREMENTS

EXAMPLE INPUT

Two selection tasks

EXPECTED OUTPUT

2:6 -> iloc; Maths > 70 -> loc

THINKING HINT

Position-based vs condition-based selection is the key distinction.

← BACK TO INDEX

LEVEL ITEM 72 • Validate a derived result

DIFFICULTY: Hard	SOURCE: s14(1).py
CONCEPTS: pandas, apply, lambda, boolean logic	SKILL: THINK

SCENARIO

You are told that Result should be pass when Total > 200. Explain how you would detect rows where the stored Result disagrees with this rule.

CHALLENGE

You are told that Result should be pass when Total > 200. Explain how you would detect rows where the stored Result disagrees with this rule.

THINKING REQUIREMENTS

- Identify what data must be stored.
- Choose the most appropriate Python concepts from the attached sources.
- State at least one edge case to handle.

REQUIREMENTS

EXAMPLE INPUT

DataFrame with Total and Result

EXPECTED OUTPUT

A boolean filter of inconsistent rows

THINKING HINT

Compute the expected result from Total and compare it to the stored Result.

← BACK TO INDEX

LEVEL ITEM 73 • Choose a clean pipeline for marks.csv

DIFFICULTY: Hard	SOURCE: marks(1).csv + s14(1).py
CONCEPTS: pandas, fillna, dropna, loc, to_csv	SKILL: THINK

SCENARIO

Design the order of operations for a reporting pipeline that must: load, clean missing data, compute Total, filter pass rows, and export the result.

CHALLENGE

Design the order of operations for a reporting pipeline that must: load, clean missing data, compute Total, filter pass rows, and export the result.

THINKING REQUIREMENTS

- Identify what data must be stored.
- Choose the most appropriate Python concepts from the attached sources.
- State at least one edge case to handle.

REQUIREMENTS

EXAMPLE INPUT

marks.csv

EXPECTED OUTPUT

A sensible ordered pipeline

THINKING HINT

Order matters: load -> clean -> derive -> filter -> save.

← BACK TO INDEX

LEVEL ITEM 74 • Explain why map/filter are not enough for a grouped report

DIFFICULTY: Hard	SOURCE: s11.py + s14(1).py
CONCEPTS: map, filter, pandas, aggregation	SKILL: Aggregation / groupby

SCENARIO

You must report the average Physics score by Result. Explain why map/filter alone are not the natural primary tool and what kind of pandas operation is more appropriate.

CHALLENGE

You must report the average Physics score by Result. Explain why map/filter alone are not the natural primary tool and what kind of pandas operation is more appropriate.

THINKING REQUIREMENTS

- Identify what data must be stored.
- Choose the most appropriate Python concepts from the attached sources.
- State at least one edge case to handle.

REQUIREMENTS

EXAMPLE INPUT

marks.csv

EXPECTED OUTPUT

A reasoned choice of aggregation

THINKING HINT

Think “group and aggregate,” not “transform each row.”

INTERVIEW / LEETCODE SKILL

Aggregation / groupby

← BACK TO INDEX

LEVEL ITEM 75 • Design a reproducible CSV audit

DIFFICULTY: Hard	SOURCE: s14(1).py + marks(1).csv
CONCEPTS: pandas, read_csv, fillna, dropna, to_csv	SKILL: THINK

SCENARIO

You need a repeatable audit that can be run every week on a fresh copy of the marks CSV. Decide which steps should be deterministic and which values should be derived rather than trusted from an input file.

CHALLENGE

You need a repeatable audit that can be run every week on a fresh copy of the marks CSV. Decide which steps should be deterministic and which values should be derived rather than trusted from an input file.

THINKING REQUIREMENTS

- Identify what data must be stored.
- Choose the most appropriate Python concepts from the attached sources.
- State at least one edge case to handle.

REQUIREMENTS

EXAMPLE INPUT

```
marks.csv
```

EXPECTED OUTPUT

```
A cleaned CSV audit file plus a short report of row counts and pass/fail totals
```

THINKING HINT

Think about the order: load, inspect, clean, derive, validate, save.

← BACK TO INDEX

NOTES

SECTION F - CONCEPT NOTES

Review these after completing the relevant question groups.

[← BACK TO INDEX](#)

Modules, imports and user-defined modules

SOURCE: s10.py

Imports bring reusable code into a file. Aliases rename imported items for local use. User-defined modules keep reusable functions separate from the main program.

IMPORTANT RULES

- Use from module import name for selected imports.
- Aliases should make code clearer, not cryptic.

COMMON MISTAKES

- Importing a function twice under confusing names.
- Putting all reusable logic in the main script.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the difference between selection vs transformation, file modes, JSON parsing vs serialization, requests vs BeautifulSoup, and loc vs iloc. Be able to read a short snippet and predict its result.

PROBLEM-SOLVING TIP

If code can be reused by multiple scripts, think module. If a main script becomes hard to read, separate helpers.

MINI CHECK

- When would you use from module import name?
- Why use an alias?

time module, sleep and regex validation

SOURCE: s10.py

sleep introduces a controlled delay. Regex checks let you validate whether a string contains required character classes.

IMPORTANT RULES

- Keep retry logic separate from validation logic.
- Regex character classes express presence tests compactly.

COMMON MISTAKES

- Off-by-one retry counters.
- Regex that checks only one requirement and ignores the rest.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the difference between selection vs transformation, file modes, JSON parsing vs serialization, requests vs BeautifulSoup, and loc vs iloc. Be able to read a short snippet and predict its result.

PROBLEM-SOLVING TIP

Break validation into independent rules, then decide how retry control should react to failure.

MINI CHECK

- What does sleep do?
- What does [0-9] mean in a regex?

List comprehensions and ternary expressions

SOURCE: s11.py

A list comprehension builds a list from an iterable in one expression. A ternary expression chooses between two values from a condition; nested ternaries should be used sparingly.

IMPORTANT RULES

- The conditional expression has true-value, condition, false-value order.
- Comprehensions should remain readable; switch to a loop when logic becomes too dense.

COMMON MISTAKES

- Writing unreadable nested ternaries.
- Confusing the expression order in a ternary.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the difference between selection vs transformation, file modes, JSON parsing vs serialization, requests vs BeautifulSoup, and loc vs iloc. Be able to read a short snippet and predict its result.

PROBLEM-SOLVING TIP

Ask whether you are building a collection or choosing between values.

MINI CHECK

- What is the order of parts in a ternary expression?
- When is a loop clearer than a comprehension?

lambda, map and filter

SOURCE: s11.py

lambda creates a small anonymous function. map transforms elements; filter keeps elements that satisfy a predicate. They can be chained when a pipeline is natural.

IMPORTANT RULES

- filter decides which items survive; map changes values.
- Convert map/filter iterators to list when a concrete list is required.

COMMON MISTAKES

- Using map when you really need selection.
- Forgetting that map/filter return iterators.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the difference between selection vs transformation, file modes, JSON parsing vs serialization, requests vs BeautifulSoup, and loc vs iloc. Be able to read a short snippet and predict its result.

PROBLEM-SOLVING TIP

Ask: keep, change, or both? That decision often tells you which functional tool to use.

MINI CHECK

- Which tool selects? Which tool transforms?
- What do map/filter return before list()?

Requests, APIs and JSON parsing

SOURCE: s12.py, s13.py

requests retrieves HTTP responses. JSON parsing turns JSON text into Python objects or serializes Python objects back to JSON text.

IMPORTANT RULES

- Parse once, then work with Python objects.
- Network/API code should account for failures in production.

COMMON MISTAKES

- Calling `json.loads` on an already-parsed Python object.
- Assuming network requests always succeed.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the difference between selection vs transformation, file modes, JSON parsing vs serialization, requests vs BeautifulSoup, and loc vs iloc. Be able to read a short snippet and predict its result.

PROBLEM-SOLVING TIP

Separate transport, parsing and transformation into distinct steps.

MINI CHECK

- What is `response.text`?
- What does `json.loads` return?

File handling

SOURCE: s12.py

`open()` selects a file mode. `read/readline/readlines` consume content; `write/append` change it. Close files after use; in production, with `open(...)` is safer.

IMPORTANT RULES

- Use the correct mode: `r`, `w`, `a`, `r+`.
- Always close files; prefer with `open(...)` in production.

COMMON MISTAKES

- Using `w` when you meant to append.
- Forgetting to close a file.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the difference between selection vs transformation, file modes, JSON parsing vs serialization, requests vs BeautifulSoup, and `loc` vs `iloc`. Be able to read a short snippet and predict its result.

PROBLEM-SOLVING TIP

Decide whether you want to read, replace, or append before opening the file.

MINI CHECK

- What is the difference between `w` and `a`?
- What does `readline` return?

Web scraping with BeautifulSoup

SOURCE: s13.py

A scraping flow is request HTML -> parse with BeautifulSoup -> find matching elements -> extract/clean text -> optionally save results.

IMPORTANT RULES

- Selectors must match the actual HTML.
- Clean text before numeric conversion.

COMMON MISTAKES

- Hard-coding a selector without checking the HTML.
- Comparing currency strings directly as numbers.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the difference between selection vs transformation, file modes, JSON parsing vs serialization, requests vs BeautifulSoup, and loc vs iloc. Be able to read a short snippet and predict its result.

PROBLEM-SOLVING TIP

Debug scraping stage-by-stage: response -> HTML -> selector -> extracted text -> cleaned value.

MINI CHECK

- What does BeautifulSoup parse?
- Why clean price strings before float()?

Pandas data loading and inspection

SOURCE: s14(1).py, marks(1).csv

`read_csv` loads tabular data into a `DataFrame`. `head/tail` inspect rows and columns reveals available fields.

IMPORTANT RULES

- Check columns before hard-coding transformations.
- `head/tail` are inspection tools, not transformations.

COMMON MISTAKES

- Assuming column names without checking `data.columns`.
- Confusing `head` and `tail`.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the difference between selection vs transformation, file modes, JSON parsing vs serialization, requests vs BeautifulSoup, and `loc` vs `iloc`. Be able to read a short snippet and predict its result.

PROBLEM-SOLVING TIP

Inspect the table before transforming it: `head`, `tail`, `columns`.

MINI CHECK

- What does `head(2)` show?
- Why inspect columns?

Pandas cleaning, transformation and filtering

SOURCE: s14(1).py, marks(1).csv

fillna replaces missing data, dropna removes incomplete rows, calculated columns derive new values, apply runs a function across Series values, loc filters by conditions, iloc selects by position, duplicated detects duplicates, drop_duplicates removes them, and to_csv persists results.

IMPORTANT RULES

- Use & and | for elementwise pandas boolean combinations.
- iloc is positional; loc is label/condition based.

COMMON MISTAKES

- Using Python and/or instead of pandas &/| for elementwise conditions.
- Treating iloc like label selection.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the difference between selection vs transformation, file modes, JSON parsing vs serialization, requests vs BeautifulSoup, and loc vs iloc. Be able to read a short snippet and predict its result.

PROBLEM-SOLVING TIP

Think in pipeline stages: load -> clean -> derive -> filter -> save.

MINI CHECK

- When use loc vs iloc?
- What does drop_duplicates do?

ANSWERS SECTION G - ANSWER KEY & EXP LANATION

Solutions for every one of the 75 questions.

[← BACK TO INDEX](#)

ANSWER 1 - Import Alias for a Function

CORRECT ANSWER

A

EXPLANATION

The source file explicitly demonstrates importing `hi_msg` with an alias and then calling `hm`.

CONCEPT NOTE

imports, aliasing, user-defined modules

[← BACK TO INDEX](#)

ANSWER 2 - `time.sleep()`

CORRECT ANSWER

A

EXPLANATION

`sleep` delays the current thread for the requested number of seconds.

CONCEPT NOTE

time module, `sleep`

[← BACK TO INDEX](#)

ANSWER 3 - Regex Password Rule

CORRECT ANSWER

A

EXPLANATION

The source uses [0-9] to test for at least one numeric character.

CONCEPT NOTE

re module, regular expressions, validation

[← BACK TO INDEX](#)

ANSWER 4 - List Comprehension Result

CORRECT ANSWER

B

EXPLANATION

Odd values are squared; even values remain unchanged.

CONCEPT NOTE

list comprehension

[← BACK TO INDEX](#)

ANSWER 5 - Ternary Operator

CORRECT ANSWER

B

EXPLANATION

The condition `age >= 18` is true.

CONCEPT NOTE

ternary operator

[← BACK TO INDEX](#)

ANSWER 6 - Nested Ternary Logic

CORRECT ANSWER

B

EXPLANATION

Negative four is even and not positive, so the nested ternary returns E and N.

CONCEPT NOTE

nested ternary, conditional expression

[← BACK TO INDEX](#)

ANSWER 7 - lambda Function

CORRECT ANSWER

B

EXPLANATION

The lambda takes one parameter and immediately returns the f-string.

CONCEPT NOTE

lambda, anonymous function

[← BACK TO INDEX](#)

ANSWER 8 - filter() Semantics

CORRECT ANSWER

B

EXPLANATION

Only values meeting $n \geq 20$ are kept.

CONCEPT NOTE

filter, lambda

[← BACK TO INDEX](#)

ANSWER 9 - map() Semantics

CORRECT ANSWER

B

EXPLANATION

Each element is squared and collected into a list.

CONCEPT NOTE

map, lambda

[← BACK TO INDEX](#)

ANSWER 10 - Chained map/filter

CORRECT ANSWER

A

EXPLANATION

The filter keeps 35, 62 and 72; map then squares them.

CONCEPT NOTE

map, filter, lambda

[← BACK TO INDEX](#)

ANSWER 11 - requests.get()

CORRECT ANSWER

B

EXPLANATION

requests.get performs a GET request and returns a Response object.

CONCEPT NOTE

requests, HTTP GET

[← BACK TO INDEX](#)

ANSWER 12 - JSON loading

CORRECT ANSWER

B

EXPLANATION

`json.loads` parses a JSON string into Python data structures.

CONCEPT NOTE

`json.loads`, JSON parsing

[← BACK TO INDEX](#)

ANSWER 13 - File mode w

CORRECT ANSWER

B

EXPLANATION

Write mode is used for writing and may truncate an existing file.

CONCEPT NOTE

open, file modes

[← BACK TO INDEX](#)

ANSWER 14 - readline()

CORRECT ANSWER

A

EXPLANATION

readline returns one line at a time.

CONCEPT NOTE

file handling, readline

[← BACK TO INDEX](#)

ANSWER 15 - json.dumps()

CORRECT ANSWER

B

EXPLANATION

dumps serializes Python data into a JSON string.

CONCEPT NOTE

json.dumps, serialization

[← BACK TO INDEX](#)

ANSWER 16 - BeautifulSoup parser

CORRECT ANSWER

B

EXPLANATION

The source uses Python's built-in HTML parser option.

CONCEPT NOTE

BeautifulSoup, web scraping

[← BACK TO INDEX](#)

ANSWER 17 - Price parsing

CORRECT ANSWER

B

EXPLANATION

Currency symbols and thousands separators are removed before numeric conversion.

CONCEPT NOTE

web scraping, string replace, float

[← BACK TO INDEX](#)

ANSWER 18 - Path(__file__).parent

CORRECT ANSWER

B

EXPLANATION

It creates a path to marks.csv relative to the script location.

CONCEPT NOTE

pathlib, Path

[← BACK TO INDEX](#)

ANSWER 19 - pandas read_csv

CORRECT ANSWER

A

EXPLANATION

read_csv returns a DataFrame from CSV data.

CONCEPT NOTE

pandas, read_csv

[← BACK TO INDEX](#)

ANSWER 20 - head() vs tail()

CORRECT ANSWER

B

EXPLANATION

head and tail inspect rows from the top and bottom respectively.

CONCEPT NOTE

pandas, head, tail

[← BACK TO INDEX](#)

ANSWER 21 - fillna vs dropna

CORRECT ANSWER

B

EXPLANATION

fillna fills missing values, while dropna removes rows containing missing values.

CONCEPT NOTE

pandas, missing data

[← BACK TO INDEX](#)

ANSWER 22 - apply lambda

CORRECT ANSWER

A

EXPLANATION

Each Total value is classified using the ternary expression.

CONCEPT NOTE

pandas, apply, lambda

[← BACK TO INDEX](#)

ANSWER 23 - loc condition

CORRECT ANSWER

A

EXPLANATION

The source demonstrates .loc with boolean conditions combined with &.

CONCEPT NOTE

pandas, loc, boolean filtering

[← BACK TO INDEX](#)

ANSWER 24 - iloc slicing

CORRECT ANSWER

A

EXPLANATION

The stop position 6 is excluded.

CONCEPT NOTE

pandas, iloc

[← BACK TO INDEX](#)

ANSWER 25 - drop_duplicates()

CORRECT ANSWER

B

EXPLANATION

drop_duplicates returns a DataFrame with duplicate rows removed.

CONCEPT NOTE

pandas, duplicates

[← BACK TO INDEX](#)

ANSWER 26 - Explain a Module Alias

SOLUTION

```
from mymodule import hello_msg as hm
```

EXPLANATION

The alias lets the function be called using a shorter/local name.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 27 - List Comprehension Rewrite

SOLUTION

```
li = [i for i in range(1, 6)]
```

EXPLANATION

A list comprehension expresses the loop-and-append pattern compactly.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 28 - Ternary Conversion

SOLUTION

```
result = "adult" if age >= 18 else "minor"
```

EXPLANATION

The ternary expression is an expression form of simple if/else logic.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 29 - filter() Task

SOLUTION

```
list(filter(lambda x: x >= 10, [4,10,15,2]))
```

EXPLANATION

The predicate selects 10 and 15.

EDGE CASES

Check when no items survive the filter and when all items survive.

[← BACK TO INDEX](#)

ANSWER 30 - Write a File

SOLUTION

```
f = open("demo.txt", "w")  
f.write("A\nB\n")  
f.close()
```

EXPLANATION

The file is opened for writing, content is written, and the file is closed.

EDGE CASES

Check missing files, empty files, wrong mode, and ensuring the file is closed.

[← BACK TO INDEX](#)

ANSWER 31 - JSON Conversion

SOLUTION

```
json.loads(text)  # JSON string -> Python  
json.dumps(obj)   # Python -> JSON string
```

EXPLANATION

These operations move between JSON text and Python data.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 32 - Scrape One Field

SOLUTION

```
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
```

EXPLANATION

requests retrieves the HTML, BeautifulSoup parses it.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 33 - Read First Two Rows

SOLUTION

```
print(data.head(2))
```

EXPLANATION

head(n) displays the first n rows.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 34 - Create Total Column

SOLUTION

```
data["Total"] = data["Maths"] + data["Physics"] + data["Chemistry"]
```

EXPLANATION

Pandas performs elementwise addition across the selected columns.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 35 - Filter Pass Rows

SOLUTION

```
data.loc[data["Result"] == "pass"]
```

EXPLANATION

Boolean filtering selects only rows whose Result is pass.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 36 - Create a reusable password validator

SOLUTION

```
import re

def valid_password(pwd):
    if len(pwd) <= 8 or len(pwd) >= 16:
        return False
    checks = [re.search(r"[A-Z]", pwd), re.search(r"[a-z]", pwd),
              re.search(r"[0-9]", pwd), re.search(r"[@$%^&*", pwd)]
    return all(checks)

print("Valid" if valid_password("Dev@1234x") else "Invalid")
```

EXPLANATION

The source uses `re.search` with separate character classes and a flag-driven loop. This version keeps the same concepts but makes the result reusable.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

COMPLEXITY

TIME: $O(n)$ SPACE: $O(1)$

← [BACK TO INDEX](#)

ANSWER 37 - Module menu wrapper

SOLUTION

```
# mymodule.py
def hello_msg(name):
    return f"Hello {name}"

def hi_msg(name):
    return f"Hi {name}"

# main.py
from mymodule import hello_msg, hi_msg
choice = 1
name = "Dev"
print(hello_msg(name) if choice == 1 else hi_msg(name))
```

EXPLANATION

The point is separation: reusable functions stay in the module, and the main file imports only what it needs.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 38 - Transform mixed values with a lambda

SOLUTION

```
nums = [3,4,7,8]
result = list(map(lambda x: x if x % 2 == 0 else x**2, nums))
print(result)
```

EXPLANATION

Each element is transformed independently using the ternary expression inside the lambda.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

COMPLEXITY

TIME: $O(n)$ SPACE: $O(n)$

[← BACK TO INDEX](#)

ANSWER 39 - Filter then transform

SOLUTION

```
nums = [3,35,62,2,72,18,9]
filtered = filter(lambda x: x >= 20, nums)
result = list(map(lambda x: x**2, filtered))
print(result)
```

EXPLANATION

The filter stage decides what survives; the map stage transforms survivors.

EDGE CASES

Check when no items survive the filter and when all items survive.

COMPLEXITY

TIME: $O(n)$ SPACE: $O(n)$

[← BACK TO INDEX](#)

ANSWER 40 - Build numbers 1-100 with comprehension

SOLUTION

```
li = [i for i in range(1, 101)]  
print(li)
```

EXPLANATION

This mirrors the source comparison between a loop-and-append version and a list comprehension.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 41 - Nested ternary classifier

SOLUTION

```
n = -8
result = ("Zero" if n == 0 else
          ("Even and Positive" if n > 0 else "Even and Negative") if n % 2 == 0 else
          ("Odd and Positive" if n > 0 else "Odd and Negative"))
print(result)
```

EXPLANATION

The exercise combines the source's nested ternary idea with an explicit zero case.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 42 - Extract API titles

SOLUTION

```
import json
raw = '[{"title":"A"}, {"title":"B"}]'
products = json.loads(raw)
titles = list(map(lambda p: p["title"], products))
print(titles)
```

EXPLANATION

The source uses json-loaded API data followed by map to extract titles.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 43 - Read a text file safely

SOLUTION

```
f = open("demo.txt", "r")
text = f.read()
f.close()
print(text)
```

EXPLANATION

This practices the exact file-opening and read() concepts shown in the source.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 44 - Append a log line

SOLUTION

```
f = open("demo.txt", "a")
f.write("Success\n")
f.close()
```

EXPLANATION

Mode a preserves existing content and adds new content at the end.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 45 - Write rows from a list

SOLUTION

```
items = ["a","b","c","d"]
f = open("demo.txt", "w")
for item in items:
    f.write(f"{item}\n")
f.close()
```

EXPLANATION

The loop writes one formatted line for each item.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 46 - Choose read method

SOLUTION

```
f = open("demo.txt", "r")
first = f.readline().rstrip("\n")
rest = [line.rstrip("\n") for line in f.readlines()]
f.close()
print(first)
print(rest)
```

EXPLANATION

readline consumes one line; readlines reads what remains.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 47 - Convert Python data to JSON

SOLUTION

```
import json
obj = [{"name": "Dev"}]
text = json.dumps(obj)
print(text)
```

EXPLANATION

`dumps` converts a Python object into JSON-formatted text.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 48 - Scrape titles from HTML

SOLUTION

```
from bs4 import BeautifulSoup
html = '<div class="product-title">Shoe A</div><div class="product-title">Shoe B</div>'
soup = BeautifulSoup(html, "html.parser")
titles = [node.text.strip() for node in soup.find_all("div", {"class":"product-title"})]
print(titles)
```

EXPLANATION

The source's scraping pattern is fetch HTML -> parse -> find_all -> read text.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 49 - Filter scraped prices

SOLUTION

```
prices = ["₹16,500", "₹17,500", "₹20,000", "₹21,000"]
for text in prices:
    value = float(text.replace("₹", "").replace(",", ""))
    if 17000 <= value <= 20000:
        print(text)
```

EXPLANATION

This follows the exact cleaning-and-filtering pattern in the source.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 50 - Save scraped titles to a file

SOLUTION

```
titles = ["A", "B", "C"]
f = open("demo.txt", "w")
for i, title in enumerate(titles, start=1):
    f.write(f"{i}.{title}\n")
f.close()
```

EXPLANATION

The source writes scraped text to a file with a manual counter; enumerate is a clean equivalent.

EDGE CASES

Check missing files, empty files, wrong mode, and ensuring the file is closed.

[← BACK TO INDEX](#)

ANSWER 51 - Build Total and Result columns

SOLUTION

```
import pandas as pd

data = pd.DataFrame({"Maths": [55, 75], "Physics": [60, 96], "Chemistry": [90, 78]})
data["Total"] = data["Maths"] + data["Physics"] + data["Chemistry"]
data["Result"] = data["Total"].apply(lambda n: "pass" if n > 200 else "fail")
print(data)
```

EXPLANATION

This is a direct exercise on the transformation steps shown in s14.py.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 52 - Handle missing values

SOLUTION

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"A": [1, None, 3], "B": [10, 20, None]})
filled = df.fillna(0)
dropped = df.dropna()
print("filled rows:", len(filled))
print("dropped rows:", len(dropped))
```

EXPLANATION

`fillna` preserves rows while replacing missing values; `dropna` removes rows containing missing data.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 53 - Inspect a DataFrame

SOLUTION

```
import pandas as pd

df = pd.read_csv("marks.csv")
print(df.head(2))
print(df.tail(2))
print(df.columns.values)
```

EXPLANATION

This mirrors the inspection block from the source file.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 54 - Filter with loc

SOLUTION

```
import pandas as pd

df = pd.read_csv("marks.csv")
selected = df.loc[(df["Maths"] > 30) & (df["Result"] == "fail")]
print(selected)
```

EXPLANATION

The source demonstrates this exact form of boolean filtering.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 55 - Combine map/filter with student totals

SOLUTION

```
import pandas as pd

df = pd.read_csv("marks(1).csv")
rows = df.to_dict("records")
qualified = filter(lambda r: r["Total"] > 210, rows)
report = list(map(lambda r: f"{r['Name']} - {r['Total']}", qualified))
print(report)
```

EXPLANATION

This intentionally combines the functional tools from s11.py with the tabular data represented in s14.py and marks.csv.

EDGE CASES

Check when no items survive the filter and when all items survive.

COMPLEXITY

TIME: $O(n)$ SPACE: $O(n)$

[← BACK TO INDEX](#)

ANSWER 56 - Password Gate for an Admin Portal

SOLUTION

```
import re
from time import sleep

def valid_password(pwd):
    if len(pwd) <= 8 or len(pwd) >= 16:
        return False
    return all([re.search(r"[A-Z]", pwd), re.search(r"[a-z]", pwd),
                re.search(r"[0-9]", pwd), re.search(r"[@$%^&*]", pwd)])

secret = "Dev@1234x"
tries = 0
for guess in ["bad", "wrong", "nope", secret]:
    if guess == secret:
        print("Welcome")
        break
    tries += 1
    if tries == 3:
        print("Sleep mode...")
        sleep(1)
        tries = 0
    else:
        print("Try again...")
```

EXPLANATION

The source combines regex validation, loop control, and sleep; together these resemble an account-protection flow.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

COMPLEXITY

TIME: $O(n)$ SPACE: Depends on output/storage

[← BACK TO INDEX](#)

ANSWER 57 - API Product Title Extractor

SOLUTION

```
import requests, json

response = requests.get("https://example.com/products")
products = response.json()
titles = list(map(lambda p: p["title"], products))
print(titles)
```

EXPLANATION

This application maps directly to the requests/API and JSON extraction pattern demonstrated in the source. Network availability would be required in a live app.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 58 - Audit trail writer

SOLUTION

```
event = "LOGIN_SUCCESS"
f = open("audit.log", "a")
f.write(event + "\n")
f.close()
```

EXPLANATION

The source explicitly distinguishes write and append modes. This is a direct real-world use of append logging.

EDGE CASES

Check missing files, empty files, wrong mode, and ensuring the file is closed.

[← BACK TO INDEX](#)

ANSWER 59 - Product Catalog Scraper

SOLUTION

```
from bs4 import BeautifulSoup

html = "<div class='product'>Shoe A</div><div class='product'>Shoe B</div>"
soup = BeautifulSoup(html, "html.parser")
titles = [x.text.strip() for x in soup.find_all("div", {"class": "product"})]

with open("product_catalog.txt", "w") as f:
    for i, title in enumerate(titles, 1):
        f.write(f"{i}.{title}\n")
```

EXPLANATION

The source shows the same pipeline: requests/HTML -> BeautifulSoup -> text extraction -> file output.

EDGE CASES

Check missing files, empty files, wrong mode, and ensuring the file is closed.

[← BACK TO INDEX](#)

ANSWER 60 - Price-band monitoring

SOLUTION

```
prices = ["₹16,500", "₹17,500", "₹20,000", "₹21,000"]
for text in prices:
    value = float(text.replace("₹", "").replace(",", ""))
    if 17000 <= value <= 20000:
        print("Match:", text)
```

EXPLANATION

This is a direct operational use of the price-cleaning and filtering logic in the source.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

← [BACK TO INDEX](#)

ANSWER 61 - Student result pipeline

SOLUTION

```
import pandas as pd

df = pd.read_csv("marks(1).csv")
df = df.fillna(0)
df["Total"] = df["Physics"] + df["Maths"] + df["Chemistry"]
df["Result"] = df["Total"].apply(lambda n: "pass" if n > 200 else "fail")
print(df[["Name", "Total", "Result"]])
```

EXPLANATION

The complete workflow is directly supported by s14.py and the supplied CSV.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 62 - Data-quality cleanup job

SOLUTION

```
def clean(df):  
    print("duplicates:")  
    print(df.duplicated())  
    df = df.drop_duplicates()  
    df = df.dropna()  
    print("final rows:", len(df))  
    return df
```

EXPLANATION

This uses only the data-cleaning operations explicitly shown in the source.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 63 - CSV segment viewer

SOLUTION

```
import pandas as pd

df = pd.read_csv("marks(1).csv")
print("rows 2:6")
print(df.iloc[2:6])
print("Maths > 70")
print(df.loc[df["Maths"] > 70])
```

EXPLANATION

The source explicitly contrasts `iloc` slicing with `loc` conditional filtering.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 64 - Exportable performance report

SOLUTION

```
import pandas as pd

df = pd.read_csv("marks(1).csv")
report = df.sort_values("Total", ascending=False)
report.to_csv("sorted_results.csv", index=False)
```

EXPLANATION

Sorting is a reasonable reporting extension using pandas plus the supplied data; persistence uses the source to_csv concept.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 65 - Functional report generator

SOLUTION

```
import pandas as pd

df = pd.read_csv("marks(1).csv")
rows = df.to_dict("records")
qualifying = filter(lambda r: r["Total"] >= 200, rows)
report = list(map(lambda r: f"{r['Name']}: {r['Result']}", qualifying))
for line in report:
    print(line)
```

EXPLANATION

This intentionally combines the functional programming concepts in s11.py with the tabular processing shown in s14.py.

EDGE CASES

Check when no items survive the filter and when all items survive.

[← BACK TO INDEX](#)

ANSWER 66 - Choose map, filter or both

SOLUTION

```
nums = [12, 7, 25, 4, 30]
result = list(map(lambda x: x**2, filter(lambda x: x >= 10, nums)))
print(result)
```

EXPLANATION

The problem forces the learner to recognize selection vs transformation and order them correctly.

EDGE CASES

Check when no items survive the filter and when all items survive.

COMPLEXITY

TIME: $O(n)$ SPACE: $O(n)$

INTERVIEW / LEETCODE CONNECTION

Filtering + transformation

[← BACK TO INDEX](#)

ANSWER 67 - Design a module boundary

SOLUTION

```
# mymodule.py
def greet(name):
    return f"Hello {name}"

def format_user(name):
    return name.strip().title()

# main.py
from mymodule import greet, format_user
name = format_user(input("Name: "))
print(greet(name))
```

EXPLANATION

The reasoning is about module cohesion and reusability, both represented by the user-defined module examples.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 68 - Make a file workflow robust

SOLUTION

```
f = open("config.txt", "r")
text = f.read()
f.close()

f = open("config.txt", "a")
f.write("new_option=true\n")
f.close()

f = open("config.txt", "r")
print(f.read())
f.close()
```

EXPLANATION

This tests the learner's understanding that file mode and file position matter and that a fresh read starts from the beginning.

EDGE CASES

Check missing files, empty files, wrong mode, and ensuring the file is closed.

[← BACK TO INDEX](#)

ANSWER 69 - Parse once, use many times

SOLUTION

```
import json
raw = '[{"title":"A","price":10},{"title":"B","price":20}]'
products = json.loads(raw)
titles = [p["title"] for p in products]
prices = [p["price"] for p in products]
print(titles)
print(prices)
```

EXPLANATION

The key insight is that `json.loads` should be done once; subsequent work is on Python objects.

EDGE CASES

Test empty input, boundary values, malformed/absent data, or missing fields where applicable.

[← BACK TO INDEX](#)

ANSWER 70 - Scraping pipeline failure points

SOLUTION

```
# Diagnostic sketch
response = requests.get(url)
print(response.status_code)
print(response.text[:200])
soup = BeautifulSoup(response.text, "html.parser")
print(len(soup.find_all("div", {"class":"product"})))
```

EXPLANATION

The source's pipeline has distinct stages: request -> response text -> parse -> selector -> extraction. Debug each stage rather than guessing.

EDGE CASES

Check missing files, empty files, wrong mode, and ensuring the file is closed.

[← BACK TO INDEX](#)

ANSWER 71 - Pick loc vs iloc

SOLUTION

```
# positional
part = df.iloc[2:6]
# conditional
strong = df.loc[df["Maths"] > 70]
```

EXPLANATION

The source explicitly explains iloc as positional slicing and loc as condition-based filtering.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 72 - Validate a derived result

SOLUTION

```
expected = df["Total"].apply(lambda n: "pass" if n > 200 else "fail")
inconsistent = df.loc[expected != df["Result"]]
print(inconsistent)
```

EXPLANATION

This tests data validation, not just data creation: derive expected values and compare against stored labels.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 73 - Choose a clean pipeline for marks.csv

SOLUTION

```
import pandas as pd

df = pd.read_csv("marks(1).csv")
df = df.fillna(0).dropna()
df["Total"] = df["Maths"] + df["Physics"] + df["Chemistry"]
passed = df.loc[df["Total"] > 200]
passed.to_csv("passed_students.csv", index=False)
```

EXPLANATION

The problem tests pipeline sequencing and the relationship between transformation and filtering.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

ANSWER 74 - Explain why map/filter are not enough for a grouped report

SOLUTION

```
summary = df.groupby("Result")["Physics"].mean()  
print(summary)
```

EXPLANATION

The attached sources teach map/filter and pandas transformation, but grouped aggregation is the natural next step for this requirement.

EDGE CASES

Check when no items survive the filter and when all items survive.

INTERVIEW / LEETCODE CONNECTION

Aggregation / groupby

[← BACK TO INDEX](#)

ANSWER 75 - Design a reproducible CSV audit

SOLUTION

```
import pandas as pd

df = pd.read_csv("marks(1).csv")
start_rows = len(df)
df = df.fillna(0).dropna()
df["Total"] = df["Maths"] + df["Physics"] + df["Chemistry"]
df["Result"] = df["Total"].apply(lambda n: "pass" if n > 200 else "fail")
print("rows in:", start_rows)
print("rows out:", len(df))
print(df["Result"].value_counts())
df.to_csv("weekly_audit.csv", index=False)
```

EXPLANATION

The reasoning challenge is to design a repeatable transformation pipeline where derived fields are recomputed from source marks and the output is persisted for later review.

EDGE CASES

Check missing values, empty selections, unexpected columns, and duplicate rows.

[← BACK TO INDEX](#)

APPENDIX

APPENDIX - QUESTION-TO-SOURCE MAP

A compact audit trail showing which source supports each question.

[← BACK TO INDEX](#)

Q	SECTION	SOURCE	CONCEPTS	DIFF
1	MCQ	s10.py	imports, aliasing, user-defined modules	Easy
2	MCQ	s10.py	time module, sleep	Easy
3	MCQ	s10.py	re module, regular expressions, validation	Medium
4	MCQ	s11.py	list comprehension	Easy
5	MCQ	s11.py	ternary operator	Easy
6	MCQ	s11.py	nested ternary, conditional expression	Medium
7	MCQ	s11.py	lambda, anonymous function	Easy
8	MCQ	s11.py	filter, lambda	Medium
9	MCQ	s11.py	map, lambda	Medium
10	MCQ	s11.py	map, filter, lambda	Hard
11	MCQ	s12.py	requests, HTTP GET	Easy
12	MCQ	s12.py	json.loads, JSON parsing	Medium
13	MCQ	s12.py	open, file modes	Easy
14	MCQ	s12.py	file handling, readline	Easy
15	MCQ	s13.py	json.dumps, serialization	Medium
16	MCQ	s13.py	BeautifulSoup, web scraping	Medium
17	MCQ	s13.py	web scraping, string replace, float	Medium
18	MCQ	s14(1).py	pathlib, Path	Medium
19	MCQ	s14(1).py	pandas, read_csv	Easy
20	MCQ	s14(1).py	pandas, head, tail	Easy
21	MCQ	s14(1).py	pandas, missing data	Medium
22	MCQ	s14(1).py	pandas, apply, lambda	Medium
23	MCQ	s14(1).py	pandas, loc, boolean filtering	Hard
24	MCQ	s14(1).py	pandas, iloc	Medium
25	MCQ	s14(1).py	pandas, duplicates	Medium
26	2-MARK	s10.py	modules, aliases	Easy
27	2-MARK	s11.py	list comprehension	Easy
28	2-MARK	s11.py	ternary operator	Easy
29	2-MARK	s11.py	filter, lambda	Easy
30	2-MARK	s12.py	file handling, write	Easy
31	2-MARK	s13.py	json.loads, json.dumps	Medium
32	2-MARK	s13.py	requests, BeautifulSoup	Medium
33	2-MARK	s14(1).py	pandas, head	Easy
34	2-MARK	s14(1).py	pandas, calculated column	Medium
35	2-MARK	s14(1).py	pandas, loc, boolean filtering	Medium

Q	SECTION	SOURCE	CONCEPTS	DIFF
36	PROBLEM	s10.py	re, regex, loop, validation	Easy
37	PROBLEM	s10.py	user-defined modules, imports, functions	Medium
38	PROBLEM	s11.py	lambda, conditional expression	Easy
39	PROBLEM	s11.py	filter, map, lambda	Easy
40	PROBLEM	s11.py	list comprehension, range	Easy
41	PROBLEM	s11.py	nested ternary, modulus, conditions	Medium
42	PROBLEM	s12.py	requests, json.loads, map	Medium
43	PROBLEM	s12.py	open, read, close	Easy
44	PROBLEM	s12.py	open, append mode	Easy
45	PROBLEM	s12.py	open, write, loop	Medium
46	PROBLEM	s12.py	read, readline, readlines	Medium
47	PROBLEM	s13.py	json.dumps, serialization	Easy
48	PROBLEM	s13.py	requests, BeautifulSoup, find_all	Medium
49	PROBLEM	s13.py	BeautifulSoup, replace, float, conditions	Medium
50	PROBLEM	s13.py	web scraping, file handling	Hard
51	PROBLEM	s14(1).py	pandas, calculated columns, apply	Medium
52	PROBLEM	s14(1).py	pandas, fillna, dropna	Medium
53	PROBLEM	s14(1).py	pandas, head, tail, columns	Easy
54	PROBLEM	s14(1).py	pandas, loc, boolean conditions	Medium
55	PROBLEM	s11.py + s14(1).py + marks(1).csv	map, filter, pandas, lambda	Hard
56	REAL	s10.py	re, validation, loop, sleep	Advanced
57	REAL	s12.py	requests, json, map	Advanced
58	REAL	s12.py	open, append, file handling	Medium
59	REAL	s13.py	requests, BeautifulSoup, file writing	Advanced
60	REAL	s13.py	BeautifulSoup, replace, float, conditions	Advanced
61	REAL	s14(1).py + marks(1).csv	pandas, fillna, dropna, apply	Advanced
62	REAL	s14(1).py	pandas, duplicated, drop_duplicates, dropna	Medium
63	REAL	s14(1).py + marks(1).csv	pandas, iloc, loc	Medium
64	REAL	s14(1).py + marks(1).csv	pandas, to_csv, sort, CSV	Hard
65	REAL	s11.py + s14(1).py + marks(1).csv	map, filter, pandas, CSV	Hard
66	THINK	s11.py	map, filter, lambda	Hard
67	THINK	s10.py	user-defined modules, imports, reusability	Hard
68	THINK	s12.py	file handling, read, append, close	Hard
69	THINK	s12.py + s13.py	json.loads, JSON, data access	Medium
70	THINK	s13.py	requests, BeautifulSoup, file handling	Hard
71	THINK	s14(1).py	pandas, loc, iloc	Medium
72	THINK	s14(1).py	pandas, apply, lambda, boolean logic	Hard
73	THINK	marks(1).csv + s14(1).py	pandas, fillna, dropna, loc, to_csv	Hard
74	THINK	s11.py + s14(1).py	map, filter, pandas, aggregation	Hard
75	THINK	s14(1).py + marks(1).csv	pandas, read_csv, fillna, dropna, to_csv	Hard